

Fast Deep Neural Network Training on Distributed Systems and Cloud TPUs

Yang You^{ID}, *Member, IEEE*, Zhao Zhang^{ID}, *Member, IEEE*, Cho-Jui Hsieh, *Member, IEEE*, James Demmel, *Fellow, IEEE*, and Kurt Keutzer, *Fellow, IEEE*

Abstract—Since its creation, the ImageNet-1k benchmark set has played a significant role as a benchmark for ascertaining the accuracy of different deep neural net (DNN) models on the image classification problem. Moreover, in recent years it has also served as the principal benchmark for assessing different approaches to DNN training. Finishing a 90-epoch ImageNet-1k training with ResNet-50 on a NVIDIA M40 GPU takes 14 days. This training requires 10^{18} single precision operations in total. On the other hand, the world's current fastest supercomputer can finish 3×10^{17} single precision operations per second (according to the Nov 2018 Top 500 results). If we can make full use of the computing capability of the fastest supercomputer, we should be able to finish the training in several seconds. Over the last two years, researchers have focused on closing this significant performance gap through scaling DNN training to larger numbers of processors. Most successful approaches to scaling ImageNet training have used the synchronous mini-batch stochastic gradient descent (SGD). However, to scale synchronous SGD one must also increase the batch size used in each iteration. Thus, for many researchers, the focus on scaling DNN training has translated into a focus on developing training algorithms that enable increasing the batch size in data-parallel synchronous SGD without losing accuracy over a fixed number of epochs. In this paper, we investigate supercomputers' capability of speeding up DNN training. Our approach is to use a large batch size, powered by the Layer-wise Adaptive Rate Scaling (LARS) algorithm, for efficient usage of massive computing resources. Our approach is generic, as we empirically evaluate the effectiveness on five neural networks: AlexNet, AlexNet-BN, GNMT, ResNet-50, and ResNet-50-v2 trained with large datasets while preserving the state-of-the-art test accuracy. Compared to the baseline of a previous study from Goyal et al. [1], our approach shows higher test accuracy on batch sizes that are larger than 16K. When we use the same baseline, our results are better than Goyal et al. for all the batch sizes (Fig. 20). Using 2,048 Intel Xeon Platinum 8160 processors, we reduce the 100-epoch AlexNet training time from hours to 11 minutes. With 2,048 Intel Xeon Phi 7250 Processors, we reduce the 90-epoch ResNet-50 training time from hours to 20 minutes. Our implementation is open source and has been released in the Intel distribution of Caffe, Facebook's PyTorch, and Google's TensorFlow. The difference between this paper and the conference-version of our work [2] includes: (1) we implement our approach on Google's cloud Tensor Processing Unit (TPU) platform, which verifies our previous success on CPUs and GPUs. (2) we scale the batch size of ResNet-50-v2 to 32K and achieve 76.3 percent accuracy, which is better than the 75.3 percent accuracy achieved in our conference paper. (3) we apply our approach to Google's Neural Machine Translation (GNMT) application, which helps us to achieve $4\times$ speedup on the cloud TPUs.

Index Terms—Fast algorithm, deep learning, parallel & distributed processing

1 INTRODUCTION

FOR deep learning applications, larger datasets and bigger models lead to significant improvements in accuracy [3], but at the cost of longer training time. For example, it takes a Nvidia M40 GPU 14 days to finish just one 90-epoch ResNet-50 training execution on the ImageNet-1k dataset. This long experiment turnaround motivates the research of training time reduction of Deep Neural Networks (DNN). The 90-epoch

ResNet-50 training requires 10^{18} single precision operations. On the other hand, according to the Nov 2018 Top 500 results, the world's current fastest supercomputer can finish 3×10^{17} single precision operations per second. So, if the training can make full use of the computing capability of the supercomputer, it should be able to finish in several seconds.

So far, the best results on scaling ImageNet-1k training have used the synchronous stochastic gradient descent method (synchronous SGD) to enable parallelism. The synchronous SGD algorithm has many inherent advantages, but at the root of these advantages is determinism (modulo floating point round-off). Determinism ensures that all valid parallel implementations of the algorithm match the behavior of the sequential version. This property is invaluable during DNN design and during the debugging of optimization algorithms. In parallel DNN training with synchronous SGD, larger batch size is important to keep up machine efficiency, as it assigns each processor sufficient amount of work in each iteration. For example, engaging 512 processors with a batch size of 1k would mean that each processor only gets a local batch of 2 images. In contrast, a larger batch size of 32k

- Y. You, J. Demmel, and K. Keutzer are with the Computer Science Division, University of California at Berkeley, Berkeley, CA 94720. E-mail: {youyang, demmel, keutzer}@cs.berkeley.edu.
- Z. Zhang is with the Texas Advanced Computing Center, University of Texas at Austin, Austin, TX 78705. E-mail: zzhang@tacc.utexas.edu.
- C.-J. Hsieh is with the Department of Computer Science, University of California at Los Angeles, Los Angeles, CA 90007. E-mail: chohsieh@cs.ucla.edu.

Manuscript received 5 Oct. 2018; revised 15 Apr. 2019; accepted 24 Apr. 2019. Date of publication 30 Apr. 2019; date of current version 11 Oct. 2019. (Corresponding author: Yang You.)

Recommended for acceptance by A. Benoit.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TPDS.2019.2913833

assigns each processor 64 images in each iteration. The latter case thus makes more efficient use of the machines, as the computation to communication ratio is higher.

Over the last two years, we have seen the focus on increasing the batch size and number of processors used in image classification training, with a resulting reduction in training time. FireCaffe [4], [5] demonstrated scaling the training of GoogleNet to 128 Nvidia K20 GPUs with a batch size of 1k and a total training time of 10.5 hours for 72 epochs. Although using a larger batch size naively can lead to significant loss in test accuracy, with the warm-up technique coupled with the linear scaling rule, Goyal et al. [1] were able to scale the training of ResNet-50 to 256 Nvidia P100 GPUs with a batch size of 8k and a total training time of 65 minutes. Using a more sophisticated approach to adapting the learning rate in the Layer-wise Adaptive Rate Scaling (LARS) algorithm [6], researchers were able to scale the batch size dramatically to 32k. On eight Nvidia P100 GPUs, they reported a 3.4 percent reduction in accuracy due to the absence of data augmentation.

Given the large batch sizes that the LARS algorithm enables, it is natural to ask: how much further can we scale out the training of DNN on the ImageNet-1k dataset? This is the investigation that led to this paper. At a high level, we find out that the 32k batch size can efficiently scale DNN training on ImageNet-1k dataset up to thousands of processors. In particular, we are able to finish the 100-epoch training with AlexNet in 11 minutes with 58.6 percent top-1 test accuracy (defined in Section 2.3) on 2,048 Intel Xeon Platinum 8160 processors. With 2,048 Intel Xeon Phi 7250 Processors, we are able to reduce the turnaround time of the 90-epoch ResNet-50 training to 20 minutes without losing accuracy, inside which the top-1 test accuracy (defined in Section 2.3) converges to 74.9 percent at 64th epoch (14 minutes from starting time).

In summary, we make the following contributions:

- We show the scaling capability of LARS up to thousands of CPUs with no loss of accuracy. Meanwhile we demonstrate that DNN can be successfully trained by CPU-based systems instead of using GPUs. We achieved the best scaling results on Intel hardware.
- We examine the generality of the LARS algorithm on both AlexNet and ResNet-50, while many other works are ResNet-50 specific. AlexNet's communication overhead is much higher than ResNet-50, which makes it hard to scale up on many machines (Section 4.4). Moreover, we found that it is hard to keep the accuracy of AlexNet when we increase the batch size (Section 3.3). Our approaches are general and can be used in all the DNN models.
- Empirically, we demonstrate that LARS is more robust than the recent work [1] at a batch size of 32K on large-scale computers. When we use the same baseline, our results are better than Goyal et al. for all the batch sizes (Fig. 20).
- Our work has been open sourced and released in the Intel distribution of Caffe, Facebook's PyTorch, and Google's TensorFlow.

The difference between this paper and the conference-version of our work [2] includes: (1) We implement our approach on Google's cloud Tensor Processing Unit (TPU) platform,

which verifies our previous success on CPUs and GPUs. (2) We scale the batch size of ResNet-50-v2 to 32K and achieve 76.3 percent accuracy, which is better than the 75.3 percent accuracy achieved in our conference paper. (3) We apply our approach to Google's Neural Machine Translation (GNMT) application, which helps us to achieve 4× speedup on the cloud TPUs.

2 BACKGROUND AND RELATED WORK

In this section, we discuss the details of data-parallel stochastic gradient descent (SGD) method, the model parallel approach, and similar work of parallelizing DNN training. There are two major directions for parallelizing DNN: data parallelism and model parallelism. All the later parallel methods are the variants of them.

2.1 Data-Parallelism SGD

In the data parallelism method, the dataset is partitioned into P parts stored on each machine, and each machine will have a local copy of the neural network and the weights (w^j). In synchronized data parallelism, the communication happens at two places: the sum of local gradients and the broadcast of the global weights. For the first part, each worker computes the local gradient ∇w^j independently, and sends the update to the master node. The master then updates $\tilde{w} \leftarrow \tilde{w} - \eta/P \sum_{j=1}^P \nabla w^j$ after it gets all the gradients from workers. Here, η is the algorithm's learning rate. For the second part, the master broadcasts $\tilde{w}$ to all workers.

There is another implementation for the communication part. The 1 reduce + 1 broadcast pattern can be replaced by 1 all-reduce operation. In this situation, each worker computes the local gradient ∇w^j independently, and then the system conducts an all-reduce operation to send the sum of the gradients ($\sum_{j=1}^P \nabla w^j$) to all the machines. After that, each machine will do the weight updating ($w^j \leftarrow w^j - \eta/P \sum_{j=1}^P \nabla w^j$) locally. In this paper, we use the all-reduce method rather than the reduce-broadcast method. This synchronized approach is a widely-used method on large-scale systems [5].

Scaling synchronous SGD to more processors has two challenges. The first is giving each processor enough useful work to do; this has already been discussed in Section 1. The second challenge is the inherent problem that after processing each local batch all processors must synchronize their gradient updates via a barrier before proceeding. This problem can be partially ameliorated by overlapping computation and communication [1], [7], but the inherent synchronization barrier remains. A more radical approach to breaking this synchronization barrier is to pursue a purely asynchronous method. A variety of asynchronous approaches have been proposed [8], [9], [10], [11]. The communication and updating rules differ in the asynchronous approach and the synchronous approach. The simplest version of the asynchronous approach is a master-worker scheme. At each step, the master only communicates with one worker. The master gets the gradients ∇w^j from the j th worker, updates the global weights, and sends the global weight back to the j th worker. The order of workers is based on first-come-first-serve strategy. The master machine is also called as *parameter server*. The idea of a parameter server was used in real-world commercial applications by the Downpour SGD approach [12], which has successfully

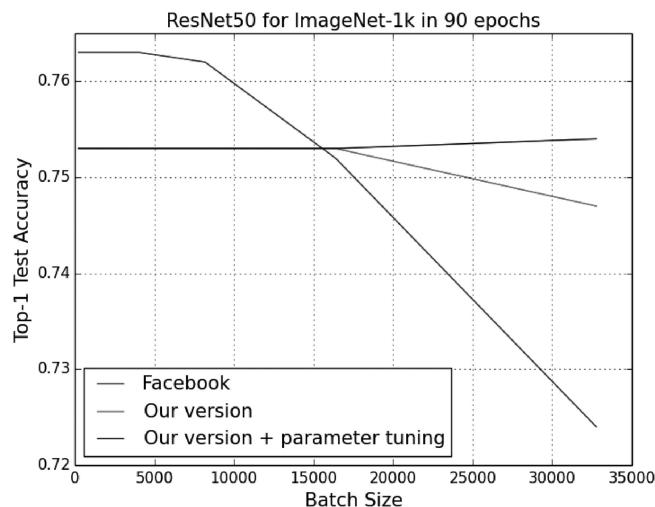


Fig. 1. Top-1 Test Accuracy Comparison on Various Batch Sizes between Our Approach and Facebook's solution.

scaled to 16,000 cores. However, Downpour's performance on 1,600 cores for a globally connected network is not significantly better than a single GPU [13].

2.2 Other Work

Goyal et al. reported finishing the 90-epoch ResNet-50 training on ImageNet-1k dataset with 256 Nvidia P100 GPUs within one hour [1]. Their work uses a batch size of 8k. Though we are able to achieve faster training speed than the reported case, our baseline's accuracy is slightly lower than Facebook's version (76.2 percent versus 75.3 percent) due to our usage of weaker data augmentation. However, our approach has a higher accuracy with batch sizes that are larger than 16k, as shown in Fig. 1. Recently, we implemented their baseline on our platform, which helps us to achieve 76.3 percent top-1 accuracy. Now our implementation is able to outperform the state-of-the-art approaches for all the batch sizes, which is shown in Fig. 20.

Codreanu et al. [14] reported achieving 73.78 percent accuracy on ResNet-50 (with data augmentation) in less than 40 minutes on 512 Intel Xeon Phi 7250 Processors. There are two things worth noting: First, their batch size is 8k. Second, Codreanu et al. only ran for 37 epochs.¹ The complete 90-epoch training would take 80 minutes with 75.25 percent accuracy.

Akiba et al. [15] reported finishing the 90-epoch ResNet-50 training within 15 minutes on 1,024 Nvidia P100 GPUs. However, the baseline accuracy is missing in the report, so it is difficult to tell how much their 74.9 percent accuracy using the 32k batch size diverges from the baseline. Second, both Akiba et al. and Goyal et al. [1] are ResNet-50 specific, while we also show the generality of our approach with AlexNet. It is worth noting that our online preprint [16] is two months earlier than Akiba et al.

2.3 Top-1 Accuracy and Top-5 Accuracy

In this section, we explain the difference between the top-1 accuracy and the top-5 accuracy. Top-1 accuracy means the

conventional accuracy: the model's prediction (the one with highest probability) must be exactly the expected answer. Top-5 accuracy means that any of your model's five highest probability predictions match the expected answer. For example, let us apply machine learning to object recognition using a neural network. A picture of an airplane is shown, and these are the outputs of our neural network:

- Car with 68 percent probability
- Train with 11 percent probability
- Bus with 10 percent probability
- Airplane with 9 percent probability
- Tank with 8 percent probability
- Gun with 2 percent probability
- Building with 1 percent probability

If we use top-1 accuracy, we count this output as false, because it predicted a car. If we use top-5 accuracy, we count this output as true, because airplane is among the top-5 guesses. In this example, the dataset has seven classes. The ImageNet-1k dataset has 1,000 classes. All the accuracy in this paper means top-1 test accuracy. Here, *test accuracy* is the prediction accuracy of the model on the validation dataset, which the model is not trained on.

2.4 Recurrent Neural Networks

Our conference publication [2] is focused on CNN (Convolutional Neural Network) applications. Another popular deep learning research direction is based on Recurrent Neural Networks or RNN [17]. RNN has been widely used in language modeling, machine translation, sentiment analysis, speech recognition, and speech synthesis [18]. A RNN layer is a matrix that can be used t times at each iteration where t is the input sequence length at runtime. Because of the vanishing and exploding gradient problems, it is hard for RNN to learn long-range dependencies. To solve this problem, the LSTM (long-short term memory) technique was proposed by Hochreiter and Schmidhuber [19] in 1997.

We picked Google's Neural Machine Translation (GNMT) or Sequence-to-Sequence [20] as our evaluating application. Our GNMT model includes the encoder part and the decoder part. The encoder has four LSTM layers with the hidden size of 1024. The first layer uses bidirectional LSTM. The other three layers use unidirectional LSTM. The residual connections start from the 3rd layer. The decoder includes four unidirectional LSTM layers with the hidden size of 1024 and a fully-connected classifier. The residual connections also start from the 3rd layer. The dataset is WMT'16 English-German translation challenge.² We use the BLEU score (higher is better) on newstest2014 dataset as the accuracy metric (BLEU score is calculated by sacrebleu package³). Our baseline model achieves a BLEU score of 21.80 [21].

2.5 Tensor Processing Unit⁴

In this section, we introduce Google's Tensor Processing Unit (TPU), which was used in our experiments. TPUv1 chip is focused on DNN inference, which was released in 2017 [22].

2. <http://www.statmt.org/wmt16/translation-task.html>

3. <https://pypi.org/project/sacrebleu/>

4. This section is based on the public information from the internet. We do not claim any contribution of this section.

1. <https://blog.surf.nl/en/imagenet-1k-training-on-intel-xeon-phi-in-less-than-40-minutes/>

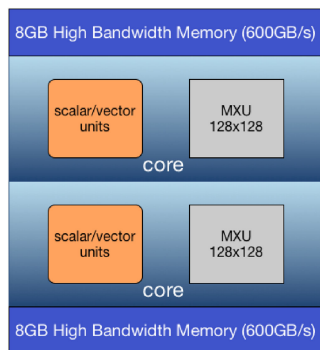


Fig. 2. The architecture of Tensor Processing Unit (TPU) chip. MXU means Matrix Unit. The performance of a TPU chip is 45 Tflops (32-bit and 16-bit mixed precision). This figure was made by ourselves based on Jeff Dean's talk at NIPS'17.

This paper is focused on DNN training, which uses TPUv2. TPUv2 was open to limited users via Google Cloud in February of 2018. This paper does not include any data on TPUv1.

TPU Chip. Fig. 2 shows the architecture of a TPU chip. Each TPU chip includes two TPU cores. Inside each TPU core, an 128-by-128 MXU (Matrix Unit) is connected to the scalar/vector units. Scalar/vector units are only for 32-bit floating point operations. MXU supports 32-bit precision for accumulation and 16-bit precision for multiplication. TPU does not support double-precision floating point operations because 32-bit operations are enough for the accuracy of machine learning applications. The performance of each TPU chip is 45 Tflops for 32-bit and 16-bit mixed-precision computation.

TPU. A TPU is made up of four TPU chips. Each TPU provides 180 Teraflops performance (32-bit and 16-bit mixed precision), 64 GB memory, and 2400 GB/s total peak bandwidth. Currently, TPU is open to the public on Google cloud. Fig. 3 is the structure of a TPU.

TPU Pod. TPU Pod is the first supercomputer for deep learning applications in the cloud (one can also run other applications on TPU Pod if the precision is enough). A TPU Pod can provide 11.5 Petaflops (32-bit and 16-bit mixed precision) performance and 4 terabytes of high-bandwidth memory. TPU Pod is open to the public in near future. Fig. 4 shows the structure of a TPU Pod.

XLA. Like GPU's cuBLAS and CPU's MKL, TPU provides Accelerated Linear Algebra (XLA) for users to build their own high-level implementations without rewriting the math kernel. XLA supports compiler, runtime, and TPU-specific optimization. Together with the basic math operations, XLA also supports the basic DL operations like convolution.

AutoGraph. If the users choose TensorFlow as the programming framework, Google cloud provides a tool called AutoGraph. Graphs are the basic form of interacting with Tensorflow and have plenty of advantages. Graphs simplify

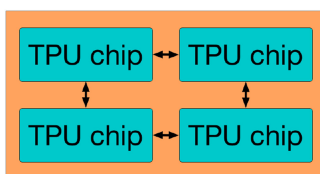
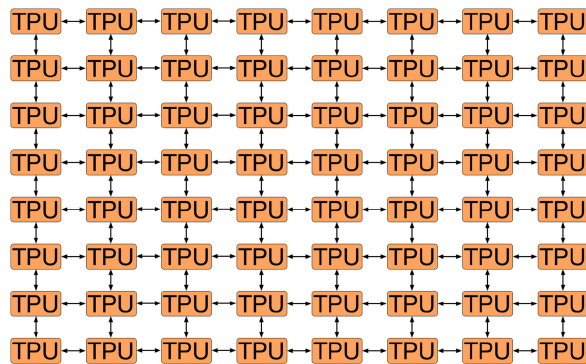


Fig. 3. Each Tensor Processing Unit (TPU) includes four TPU chips. The performance of a TPU is 180 Tflops. This figure was made by ourselves based on Jeff Dean's talk at NIPS'17.



TPU Pod: 11.5 petaflops

Fig. 4. Each TPU Pod is made up of 64 second generation TPUs. The performance of a TPU Pod is 11.5 Petaflops (32-bit and 16-bit mixed precision). This figure was made by ourselves based on Jeff Dean's talk at NIPS'17.

deployment to all sorts of environments, as they can be a platform-independent view of the application's computation. In a dataflow graph, dependencies are explicit which makes it relatively easy to parallelize and distribute the computation. Graphs also allow for whole-program optimizations like kernel fusion which replaces a subgraph with an optimized version by combining nodes. However, the programming model for building graphs in Python is complex and unintuitive. It is like constructing the Abstract Syntax Tree for the program by hand. When it comes to control flow, programmers have to implement it using an unusual functional style. On the other hand, TensorFlow's Eager execution is an imperative programming environment that evaluates operations immediately without building graphs. The users can actually combine the Eager mode and the Graph mode within a single project. The users can call from Eager into graph mode for extra performance. For example, one can implement an especially tricky algorithm in Eager and call it from graph mode. Graph code invoked from Eager is subject to the same clunky interfaces, and Eager code run from the graph is a black box to any graph-based optimizations. It is also not something that can currently be done on a TPU. So these two modes can be combined, but the integration is not perfect. And they can be used at different points of the project, but translating from one to another can be tricky. This is where AutoGraph comes in. AutoGraph allows the users to write Eager-style code and generate graphs anyway. AutoGraph can take a function written using standard Python syntax (that can run in Eager mode) and generate the code that creates a graph performing the same computation. AutoGraph will automatically convert nice Python constructs like if statements and loops into tf.conds and tf.while_loops. The users just annotate the functions they want to be converted with AutoGraph. In summary, AutoGraph helps users more easily express complex models in TensorFlow without sacrificing the benefits of graph mode such as performance and portability. The users can think of AutoGraph as adding another stage of computation. Without AutoGraph graph-style Python code is translated into graphs which can be executed by TensorFlow. With AutoGraph users write Eager-style code, autograph generates the graph style code, and TensorFlow executes the resulting graph.

Profiling. The TPU users are able to do profiling by TensorBoard, where they can easily check CPU usage, TPU usage,

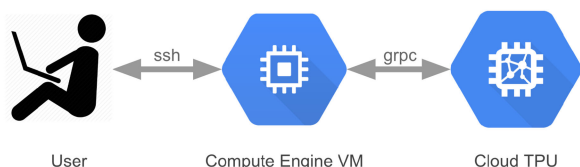


Fig. 5. This figure illustrates how a regular user can use a cloud TPU. Cloud TPUs are network-attached. Google cloud does not require the user to install any driver. The user can just use the machine images provided by Google cloud.

and the efficiency of Tensorflow/XLA XLA ops. Another useful tool is CTPU (The Cloud TPU Provisioning Utility), which allows the users to configure cloud TPUs and monitor them at runtime.

Using TPUs. Fig. 5 illustrates how a regular user can get access to a cloud TPU. Cloud TPUs are network-attached. The user only needs to create a compute engine virtual machine and apply a cloud TPU. The virtual machine connects to the cloud TPU through grpc. The regular users do not need to install any driver and can just use the machine images provided by Google cloud. However, the users still need to design the algorithm and write the code for their own applications. TPUs only provide the basic linear algebra and machine learning functions. In this way, using a cloud TPU is similar to using a GPU or FPGA.

3 LARGE BATCH DNN TRAINING

In this section, we discuss the benefits and challenges of large batch training, and the rationale of our model selection along with a learning rate profile study. Throughout the discussion, we focus on the data-parallel synchronous SGD approach, as it is proven to be stable for DNN training at scale [1]. In contrast, the asynchronous methods using parameter server are not guaranteed to be stable in a distributed environment [23].

3.1 Benefits of Large Batch Training

The prominent advantage of large batch training is that it can reduce the overall training time. The idea is straightforward—by using a large batch size for SGD, the work for each iteration can be distributed to multiple processors. Consider the following ideal case. ResNet-50 requires 7.72 billion single-precision operations to process one 225x225 image. If we run 90 epochs on the ImageNet-1k dataset of 1.28×10^6 images, the total number of operations is $90 * 1.28 \times 10^6 * 7.72 \times 10^9 \approx 10^{18}$. Currently, the world's current fastest supercomputer can finish 3×10^{17} single precision operations

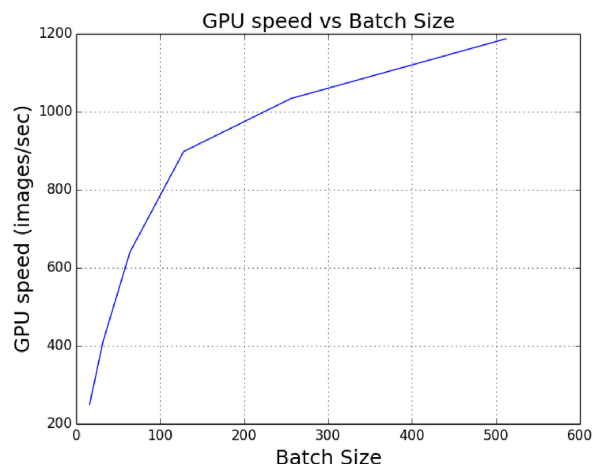


Fig. 6. AlexNet training performance on various batch sizes on a Nvidia M40 GPU. Peak performance is reached with the batch size of 512, while a 1,024 batch size runs out of memory.

per second (according to the Nov 2018 Top 500 results). If there is an algorithm allowing us to make full use of the supercomputer, we can finish the training in several seconds.

To do so, we need to make the algorithm use more processors and load more data at each iteration, which corresponds to increasing the batch size in synchronous SGD. Ideally, if we fix total number of data accesses and grow the batch size linearly with number of processors, the number of SGD iterations will decrease linearly and the time cost of each iteration remains constant, so the total time will also reduce linearly with number of processors. A detailed analytical study on the ResNet-50 training is shown in Table 1.

In the strong scaling situation, a large batch does not change the number of floating point operations (computation volume), as the number of epochs is fixed. However, a large batch can reduce the overall communication volume. The reason is that a larger batch size results in fewer iterations and so less overall communication volume, as the single iteration communication volume remains relatively constant given the fact that it is only related to the model size and the networking system. In this way, a large batch size reduces the overall DNN training time in a scalable manner.

A second benefit of large batch training is that it can keep up the high machine utilization, which is especially important in a distributed environment.

Let us use one Nvidia M40 GPU to illustrate this benefit on a single machine. Fig. 6 shows the M40 GPU performance measurements (in images/sec) for AlexNet with varying

TABLE 1
An Analytical Scaling Performance Study with ResNet-50 as the Example

Batch Size	Num of Epochs	Num of Iterations	Num of GPUs	Single Iteration Time	Total Time
512	100	250k	1	t_1	$250kt_1$
1024	100	125k	2	$t_1 + \log(2)t_2$	$125k(t_1 + \log(2)t_2)$
2048	100	62500	4	$t_1 + \log(4)t_2$	$62500(t_1 + \log(4)t_2)$
4096	100	31250	8	$t_1 + \log(8)t_2$	$31250(t_1 + \log(8)t_2)$
8192	100	15625	16	$t_1 + \log(16)t_2$	$15625(t_1 + \log(16)t_2)$
...	...	...	...	...	...
1.28M	100	100	2500	$t_1 + \log(2500)t_2$	$100(t_1 + \log(2500)t_2)$

t_1 is the computation time and t_2 is communication time. We fix the number of epochs as 100. Larger batch size needs less iterations. We set batch size as 512 per machine. Then we increase the number of machines. Since $t_1 \gg t_2$ for using ImageNet-1k dataset to train ResNet-50 on GPUs [1], the single iteration time is dominant by the computation. Thus the total time will be reduced approximately linearly.

Authorized licensed use limited to: Peking University. Downloaded on July 13, 2026 at 19:07:08 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Scaling Ratio for State-of-the-Art Models

Model	comm: parameters	comp: flops per image	comp/comm scaling ratio
AlexNet	61 million	1.5 billion	24.6
ResNet50	25 million	7.7 billion	308
VGG-19	144 million	120 billion	855
GoogleNet	13.5 million	9.7 billion	736

batch size from 16 to 512. Increasing the batch size from 16 to 32, the performance almost doubles. And from 128 to 512, the curve flattens, which means the M40 GPU approaches its peak performance at the batch size of 512. A large batch size, such as 8,192, can keep a 16 M40 GPU cluster at its peak performance during the training execution.

3.2 Model Selection

To scale out DNN training to many machines, a major overhead is the communication among different machines [24]. Here we define the notion of *scaling ratio* as ratio between computation and communication. For DNN models, the computation is proportional to the number of floating point operations required for processing an image. Since we focus on synchronous SGD approach, the communication is proportional to model size (or the number of parameters). Different DNN models have different scaling ratios. To generalize our study, we pick four representative models: AlexNet, GoogleNet, ResNet-50, and VGG. The reason is that they have different scaling ratios. From Table 2, we see that ResNet-50's scaling ratio is 12.5 $\times$ larger than that of AlexNet. This means scaling ResNet-50 is easier than scaling AlexNet. Our experiments in Figs. 15, 16, 17, and 18 confirmed this conclusion.

For VGG-19, the model size is 575MB (144 million parameters). The model requires around 120 billion operations to process one image. Thus, the scaling ratio is around 855. For GoogleNet, the model size is 54MB (13.5 million parameters). The model requires around 9.7 billion operations to process one image. Thus, the scaling ratio is around 736. In summary, the scaling ratio comparison is: VGG-19 > GoogleNet > ResNet-50 > AlexNet. Although VGG-19 has the best scaling ratio, it is slow to train because it needs many more operations than other models. For the image classification accuracy: ResNet-50 > GoogleNet > VGG-19 > AlexNet.

3.3 Challenges of Large Batch Training

Large batch size comes with the benefits of shorter training time and high machine utilization. However, naively using synchronous SGD with a large batch size usually suffers test accuracy degradation compared to smaller batch sizes with a fixed number of epochs. Unfortunately, there is no algorithm allowing us to effectively use unlimitedly large batch

TABLE 4
AlexNet Test Accuracy with varying Batch Size

Batch Size	Base LR	warmup	epochs	test accuracy
512	0.02	N/A	100	0.583
1k	0.02	no	100	0.582
4k	0.01	yes	100	0.509
4k	0.02	yes	100	0.527
4k	0.03	yes	100	0.520
4k	0.04	yes	100	0.530
4k	0.05	yes	100	0.531
4k	0.06	yes	100	0.516
4k	0.07	yes	100	0.001
...	...	...	...	...
4k	0.16	yes	100	0.001

Current approaches (linear scaling + warmup) do not work for AlexNet with a batch size larger than 1k. We tune the warmup epochs from 0 to 10 and pick the one with highest accuracy. According to linear scaling, the optimal learning rate (LR) of batch size 4k should be 0.16. We use the poly learning rate policy, and the poly power is 2. The momentum is 0.9 and the weight decay is 0.0005.

sizes [25]. Table 3 shows the target test accuracy by standard benchmarks. For example, when we set the batch size of AlexNet larger than 1,024 or the batch size of ResNet-50 larger than 8,192, the test accuracy will be significantly degraded, as shown in Table 4 and Fig. 8, respectively.

For large batch training, it is essential to keep up the test accuracy with smaller batches under the constraint of the same number of epochs. Here we fix the number of epochs because: statistically, one epoch means the algorithm touches the entire dataset once; and computationally, fixing the number of epochs means fixing the number of floating point operations. State-of-the-art techniques for large batch training to remedy the test accuracy degradation issue include:

- (1) *Linear Scaling* [27]: With an increase of the batch size from B to kB , we should also increase the learning rate from η to $k\eta$.
- (2) *Warmup Scheme* [1]: With a large learning rate (η), we should start from a small η and increase it to the large η in the first few epochs.

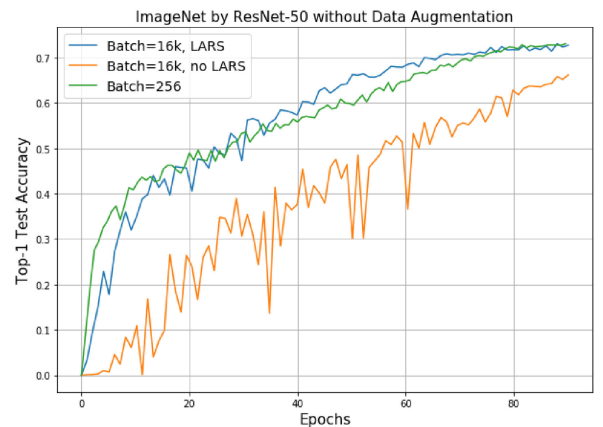


Fig. 7. *Batch Size=16k*. Test accuracy comparison between large-batch training, large batch training with LARS, and the baseline. The base learning rate of batch 256 is 0.2 with poly policy (power=2). For the version without LARS, we use the state-of-the-art approach [1]: 5-epoch warmup and linear scaling for LR. For the version with LARS, we also use 5-epoch warmup. Clearly, the existing method does not work for batch size larger than 8K. LARS algorithm can help the large batch to achieve the same accuracy with baseline in the same number of epochs.

TABLE 3
Standard Benchmarks for ImageNet-1k Training

Model	Epochs	Test Top-1 Accuracy
AlexNet	100	58% [5]
ResNet-50	90	75.3% [26]
ResNet-50-v2	90	76.3% [1]

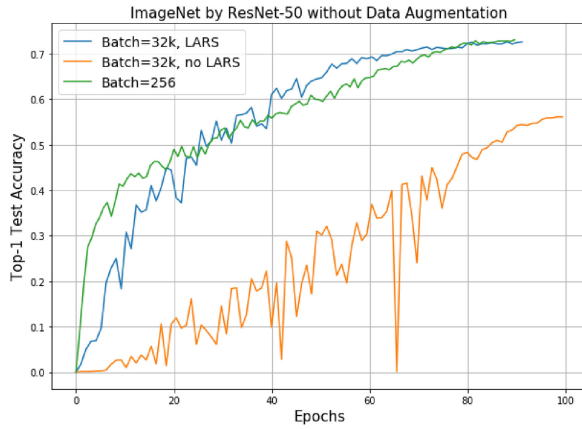


Fig. 8. *Batch Size=32k*. Test accuracy comparison between large batch training, large batch training with LARS, and the baseline. The base learning rate of batch 256 is 0.2 with poly policy (power=2). For the version without LARS, we use the state-of-the-art approach [1]: 5-epoch warmup and linear scaling for LR. For the version with LARS, we also use 5-epoch warmup. Clearly, the existing method does not work for batch size larger than 8K. LARS algorithm can help the large batch to achieve the same accuracy with baseline in the same number of epochs.

The intuition of linear scaling is related to the number of iterations [1], [27]. Let us use B , η , and I to denote the batch size, the learning rate, and the number of iterations. If we increase the batch size from B to kB , then the number of iterations is reduced from I to I/k . This indicates that the frequency of weight updating is reduced by k times. Thus, we make the updating of each iteration $k\times$ more efficient by enlarging the learning rate by k times. The purpose of a warmup scheme is to avoid the situation in which the algorithm diverges at the beginning because we have to use a very large learning rate based on linear scaling. With these techniques, researchers can use the relatively large batch in a certain range (Table 5). However, we observe that these state-of-the-art approaches can only scale batch size to 1k for AlexNet and 8k for ResNet-50. With the batch size of 4k for AlexNet, we can only achieve a 53.1 percent test accuracy in 100 epochs (Table 4). Our target is to preserve the 58 percent test accuracy even when using large batch sizes, such as 32k.

3.4 Scaling up Batch Size

To improve the accuracy for large batch training, a new rule of learning rate (LR) schedule was developed [6]. As discussed in Section 2.1, we use $w = w - \eta \nabla w$ to update the weights. Each layer has its own weight w and gradient ∇w . Standard SGD algorithm uses the same LR (η) for all the layers. However, from our experiments, we observe that different layers may need different LR. The reason is that the ratio between $\|w\|_2$ and $\|\nabla w\|_2$ varies significantly for different layers. For example, we observe that $\|w\|_2/\|\nabla w\|_2$ is only 20 for conv1.1

TABLE 5
State-of-the-art Large Batch Training and Test Accuracy

Team	Model	Batch1	Batch2	Acc1	Acc2
Google [27]	AlexNet	128	1024	57.7%	56.7%
Amazon [28]	ResNet101	256	5120	77.8%	77.8%
Facebook [1]	ResNet50	256	8192	76.30%	76.26%

Batch1 means baseline batch size. Batch2 means large batch size. Acc1 means baseline accuracy. Acc2 means large-batch accuracy.

TABLE 6
The Ratios between $\|w\|_2$ and $\|\nabla w\|_2$ for Different Layers of AlexNet with Batch Size = 4k at the 1st Epoch

Layers	$\ w\ _2$	$\ \nabla w\ _2$	$\ w\ _2/\ \nabla w\ _2$
fc8.0	20.24	0.078445	258
fc8.1	0.316	0.006147	51
fc7.0	20.48	0.110949	184
fc7.1	6.400	0.004939	1296
fc6.0	30.72	0.097996	314
fc6.1	6.400	0.001734	3690
conv5.0	6.644	0.034447	193
conv5.1	0.160	0.000961	166
conv4.0	8.149	0.039939	204
conv4.1	0.196	0.000486	403
conv3.0	9.404	0.049182	191
conv3.1	0.196	0.000511	384
conv2.0	5.545	0.057997	96
conv2.1	0.160	0.000649	247
conv1.0	1.866	0.071503	26
conv1.1	0.098	0.004909	20

We observe that they are very different from each other. *fc* is the fully connected layer and *conv* is the convolutional layer. *x.0* is a layer's weight. *x.1* is a layer's bias.

layer (Table 6). However, $\|w\|_2/\|\nabla w\|_2$ is 3,690 for fc6.1 layer. To speed up the convergence for fc6.1 layer, the users need to use a large LR. However, this large LR may lead to divergence on the conv1.1 layer. We believe this is an important reason for the optimization difficulty in large batch training.

Goyal et al. [1] proposed the warmup scheme to solve this problem. The warmup scheme works well for ResNet-50 training with a batch size $\leq 8k$. However, only using this recipe does not work for AlexNet with batch size $> 10k$ and ResNet-50 with batch size $> 8k$.

Together with researchers at Nvidia, we proposed the Layer-wise Adaptive Rate Scaling (LARS) algorithm [6] to improve large batch training's test accuracy. The base LR rule is defined in Equation (1). l is the scaling factor, which we set as 0.001 for AlexNet and ResNet training. γ is a tuning parameter for users. Usually γ can be chosen by linear scaling.

$$\eta = l \times \gamma \times \frac{\|w\|_2}{\|\nabla w\|_2}. \quad (1)$$

In this formulation, different layers can have different LR. In practice, we add momentum (denoted as μ) and weight decay (denoted as β) to SGD, and use the following sequence for LARS:

- (1) get the local LR for each learnable parameter by $\alpha = l \times \|w\|_2/(\|\nabla w\|_2 + \beta\|\nabla w\|_2)$;
- (2) get the LR for each layer by $\eta = \gamma \times \alpha$;
- (3) update the gradients by $\nabla w = \nabla w + \beta w$;
- (4) update acceleration term a by $a = \mu a + \eta \nabla w$;
- (4) update the weights by $w = w - a$.

Using this approach together with the warmup technique, SGD with a large batch can achieve identical test accuracy with the batch size of 32k as the baseline for AlexNet (Table 7). Technically, we change the local response normalization (LRN) to batch normalization (BN). We add BN after each convolutional layer. As shown in Fig. 7 and Fig. 8, we can see that the LARS algorithm can keep up the test accuracy for ResNet-50 using 32k batch with the baseline of ~ 73 percent without data augmentation. In comparison,

TABLE 7
Test Accuracy of AlexNet with Batch Size of 32k using KNL
Nodes on Stampede2

Batch Size	LR rule	warmup	Epochs	test accuracy
512	regular	N/A	100	0.583
4096	LARS	13 epochs	100	0.584
8192	LARS	8 epochs	100	0.583
32768	LARS	5 epochs	100	0.585

We use poly learning rate policy, and the poly power is 2. The momentum is 0.9 and the weight decay is 0.0005. For a batch size of 32K, we changed local response norm in AlexNet to batch norm. Specifically, we use the refined AlexNet model by Boris Ginsburg.

the current approaches of combining linear scaling and warmup has lower accuracy on ResNet-50 for batch size of 16k and 32k (68 and 56 percent, respectively).

4 PERFORMANCE EVALUATION

In this section, we evaluate the 100-epoch AlexNet and 90-epoch ResNet-50 training on a number of platforms. We will briefly introduce the hardware and software settings, and present the comparison baseline and code change. In addition to performance results, we will also discuss the communication overhead analysis.

4.1 System Configuration

Throughout this section, we run experiments on four types of hardware. The Intel Xeon Phi 7250 Processors and the Xeon Platinum 8160 processors are part of the Stampede2 super-computer hosted at Texas Advanced Computing Center.⁵ The eight Nvidia P100 GPU server is locally hosted. We also use the cloud TPUs.

We perform the large batch scaling efficiency study on the eight Nvidia P100 GPU server. Each P100 GPU has the performance of 10.6 teraflops and has 16 GB memory. The Intel Xeon Phi 7250 Processor (referred to as KNL) is the latest version of Intel's general-purpose accelerator. It is a self-hosted platform running CentOS 7 on our testbed. Each processor has 68 physical cores, and four hardware threads per core. All cores are running at 1.4 GHz clock rate. On Stampede2, 3,696 out of the total 4,200 KNL nodes are configured in the following way: On each node, there is one processor with 96 GB DDR4 RAM, 16 GB MCDRAM, and a 200 GB local Solid State Drive, of which 144 GB is available. The memory is configured as cache-quadrant mode, where MCDRAM is used as an L3 cache. The Intel Xeon Platinum 8160 processors (referred to as SKX) are part of Intel Xeon Scalable Processors collection. Each SKX node in Stampede2 has two such processors with 48 physical cores in total. Each core is documented to have a 2.1 GHz clock rate, however, the clock rate varies from 1.4 GHz to 3.7 GHz depending on the instruction set and the number of active cores. Our measured runtime clock rate for AlexNet and ResNet-50 are 2.1 GHz and 2.0 GHz, respectively. Each SKX node is equipped with 192 GB RAM and a 200 GB Solid State Drive, of which 144 GB is available. There are 1,600 SKX nodes (3,200 processors) on Stampede2.

On Nvidia GPUs, we used the Nvidia distribution of Caffe.⁶ And on Intel processors, we used two variants of the

TABLE 8
Communication Unit is Much Slower than Computation unit
because Time-Per-Flop (γ) $\ll$ 1/ Bandwidth (β) $\ll$ Latency (α)

Network	α (latency)	β (1/bandwidth)
Mellanox 56Gb/s FDR IB	0.7×10^{-6} s	0.2×10^{-9} s
Intel 40Gb/s QDR IB	1.2×10^{-6} s	0.3×10^{-9} s
Intel 10GbE NetEffect NE020	7.2×10^{-6} s	0.9×10^{-9} s

For example, $\gamma = 0.9 \times 10^{-13}$ s for NVIDIA P100 GPUs.

official Caffe [29]: 1) our customized parallel implementation that uses MPI [30] for the communication across nodes and 2) the Intel distribution of Caffe v1.0.3,⁷ which supports multi-node training by Intel Machine Learning Scaling Library (MLSL) v2017.1.016.⁸

4.2 Data and Baseline

Throughout the performance evaluation, we use the ImageNet-1k [31] dataset. The dataset has 1.28 million images for training and 50,000 images for testing. There are two top-1 test accuracy baselines for ResNet-50 in 90 epochs: the case without data augmentation is 73 percent while the case with data augmentation is 75.3 percent. The top-1 test accuracy baseline for AlexNet in 100 epochs is about 58 percent. We also got the same ResNet-50 baseline used by Goyal et al. [1]. The learning rate is 0.1 for a batch size of 256. The weight decay is $1e-4$. The momentum is 0.9. The batch norm decay is 0.9. The batch norm epsilon is $1e-5$. The label smoothing parameter used in the softmax-cross-entropy is 0.0. We got 76.3 percent top-1 accuracy from this baseline. The clipping threshold for the trust-ratio of LARS solver is [0.0, 50.0]. The clipping threshold for the gradient of LARS solver is [-10.0, 10.0].⁹

4.3 Data Shuffling in Distributed Training

For the extremely large dataset, our approach is to partition the full data set on all nodes and configure the data layer to draw a different subset on each node. Since each node has its own unique randomizing seed in this situation, it will effectively draw a unique image subset. If the dataset is not large (e.g., ImageNet), we just copy the full dataset on all nodes. Like Goyal et al., we use a single random shuffling of the training data (per epoch) that is divided amongst all the nodes.

4.4 Scaling Efficiency of Large Batches

As discussed in Section 3.1, using large batch size can reduce the communication volume with fewer iterations, thus yielding higher scaling efficiency than small batch size. Here, we present an analytical study and the empirical performance evaluation to validate this hypothesis.

Communication often is the major bottleneck for efficient scaling for applications across many processors (Table 8). On a distributed system, communication means moving the data over the network (e.g., master machine broadcast its data to all the worker machines). In DNN training, communication across nodes is in the form of a all-reduce (sum of local gradients). These communication patterns have a higher scaling

7. <https://github.com/intel/caffe>

8. <https://github.com/intel/MLSL>

9. https://people.eecs.berkeley.edu/~youyang/lars_optimizer

https://www.cs.berkeley.edu/~youyang/lars_optimizer.py

5. portal.tacc.utexas.edu/user-guides/stampede2

6. <https://github.com/NVIDIA/caffe>

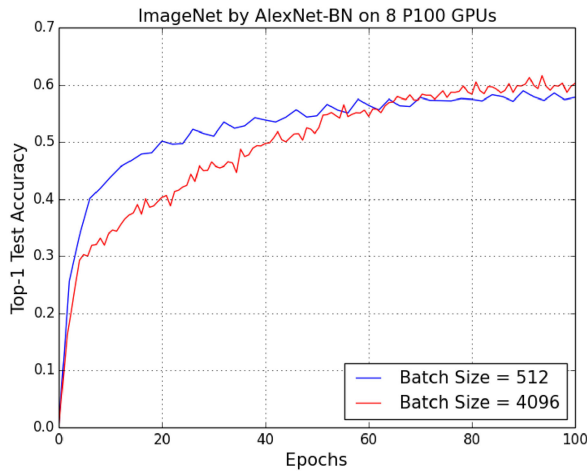


Fig. 9. Test accuracy comparison between the small batch size and the large batch size. The 512 small batch size is the baseline.

overhead than the matrix computations (i.e., the matrix computations on each machine can be finished independently). In particular, the all-reduce on N nodes have the scaling factor of $O(\log N)$ or $O(N)$ depending on the network topology [32], [33], [34], [35]. The scaling factor of broadcast is $O(\log N)$. In contrast, the matrix computation in DNN training can be distributed almost evenly to N nodes with the scaling factor of $O(1/N)$.

For finishing the same number of epochs, the communication overhead is lower in the large batch version than in the small batch version, as the large batch version sends fewer messages (latency overhead) and moves less data (bandwidth overhead). For synchronous SGD, the algorithm needs to conduct an all-reduce operation (sum of gradients on all machines) in each iteration. The number of messages sent is linear with the number of iterations. And for each iteration, the communication volume is constant regardless of the batch size, as the gradients have identical size as the model weights ($|W|$).

Let us use the following notations for the analytical evaluation:

E the number of epochs

n the total number of images in the training dataset

B the batch size

Then the number of iterations is $E \times n/B$. Holding E and n constant, with the large batch size, the program finishes with fewer iterations. By fixing E , the number of epochs, it is fixing the total number of floating point operations. Meanwhile, the number of iterations is consistent with the communication frequency of the training process. Let us denote $|W|$ as the neural network model size. Then we can get the communication volume is $|W| \times E \times n/B$.

Thus, the large batch version transfers less data than the small batch version to finish the same number of floating point operations. As mentioned before, the number of floating point operations remain constant when the number of epochs is fixed. The larger batch size increases the computation-communication ratio because it reduces the communication frequency. As a result, the larger batch size makes the algorithm more scalable on distributed systems.

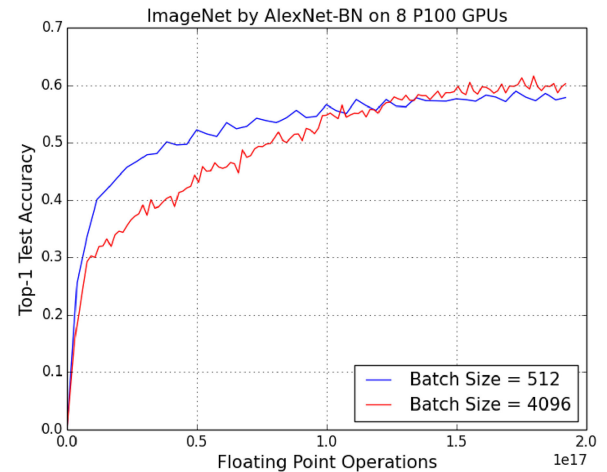


Fig. 10. Increasing the batch size does not increase the number of floating point operations. Large batch can achieve the same accuracy in the fixed number of floating point operations.

For the empirical performance evaluation, we use the ImageNet-1k training with AlexNet-BN on eight P100 GPUs in this experiment. Here, AlexNet-BN means changing the local response normalization in AlexNet to batch normalization. The baseline's batch size is 512 and is referred to as the small batch size. The large batch size is 4k. In this example, we focus on the the communication across GPUs. First, the experimental results confirm that the large batch size achieves the same test accuracy as the small batch size in 100 epochs (Fig. 9). Thus, large batch size achieves the same test accuracy as the small batch size in fixed number of floating point operations (Fig. 10).

We observed a 3x reduction in training time with the large batch size compared to that of the small batch size, as shown in Fig. 11. The number of iterations in large-batch training is much less than small-batch training (Fig. 12). The number of messages is equal to the number of iterations. Our experimental results also confirmed that large batch will reduce the accumulated latency overhead (Fig. 13) and bandwidth overhead (Fig. 14).

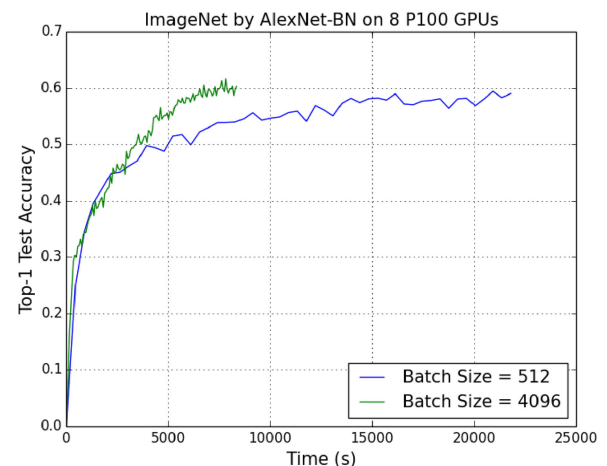


Fig. 11. Time-to-solution Comparison between the Small Batch Size and the large Batch Size. To achieve the 58 percent accuracy, the large batch size of 4k only needs about two hours while the smaller batch size of 512 needs about six hours.

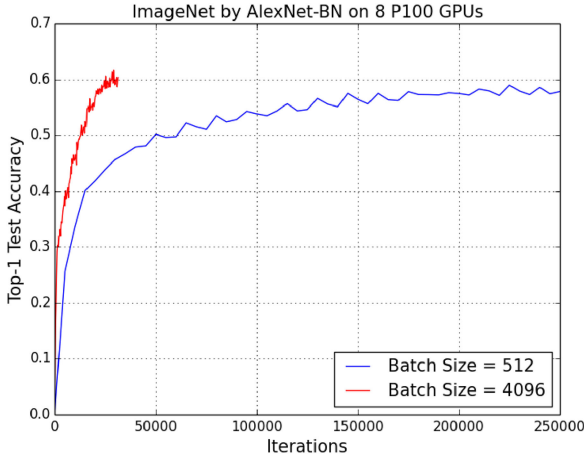


Fig. 12. When we fix the number of epochs and increase the batch size, we need much fewer iterations.

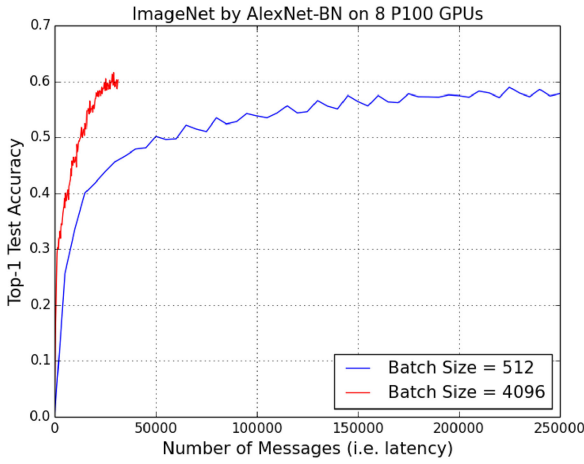


Fig. 13. When we fix the number of epochs and increase the batch size, we need much fewer iterations. The number of iterations is linear with the number of messages the algorithm sent.

4.5 ImageNet training with AlexNet

In this experiment, we use the AlexNet training case to evaluate our approach's effectiveness in scaling DNN training at

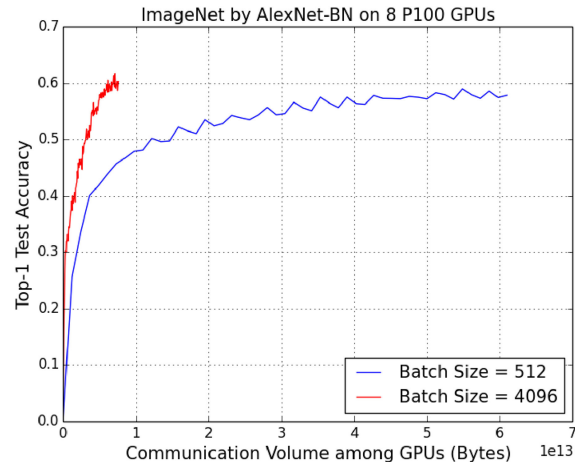


Fig. 14. The number of iterations is linear with the number of messages the algorithm sent. Let us denote $|W|$ as the neural network model size. Then we can get the communication volume is $|W| \times E \times n/B$. Thus, the larger batch version needs to move much less data than the smaller batch when they finish the number of floating point operations.

Authorized licensed use limited to: Peking University. Downloaded on July 13, 2026 at 19:07:08 UTC from IEEE Xplore. Restrictions apply.

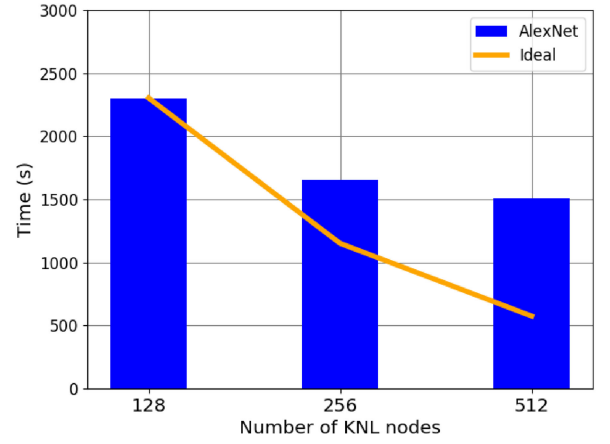


Fig. 15. Strong scaling performance of AlexNet with 32k batch size on KNL nodes.

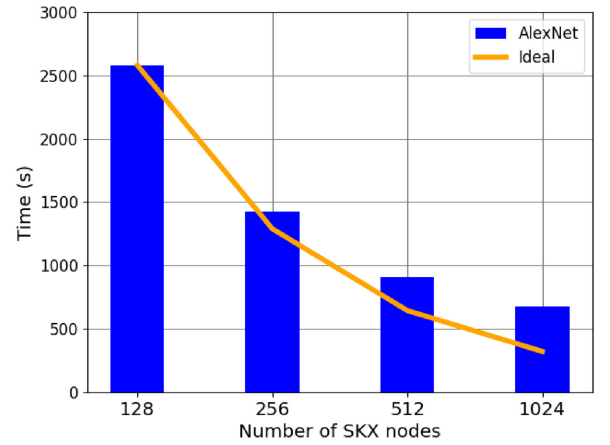


Fig. 16. Strong scaling performance of AlexNet with 32k batch size on SKX nodes.

large scale. Previously, Nvidia reported that using one DGX-1 station they were able to finish 90-epoch ImageNet-1k training with AlexNet in two hours.¹⁰ However, they used half-precision or FP16, whose cost is half of the standard single-precision operation. We run the AlexNet training with standard single-precision. It takes 6 hours 9 minutes with the batch size of 512 on one NVIDIA DGX-1 station. With our approach, using the large batch size of 4k achieves similar test accuracy as the small batch size case (Line 2 in Table 7), and it finishes in two hours and ten minutes on the same station. Thus, using large batch size can significantly speedup DNN training on GPU cluster.

Then we scale the same AlexNet training case with a batch size of 32k, and run it on multiple scales of the KNL nodes and the SKX nodes on the Stampede2 supercomputer. Figs. 15 and 16 show the strong scaling performance on each type of nodes with the ideal scaling curve relative to the performance of 128 nodes in each case.

Despite the 11-minute training time on 1,024 SKX nodes, the AlexNet training does not scale well beyond 512 nodes in both cases. The inefficient scaling performance is due to its low scaling ratio, as defined in Section 3.2. On the other hand, on 512 KNL nodes, the AlexNet training finished in 24 minutes.

10. www.nextplatform.com/2016/04/06/dgx-1-nvidias-deep-learning-system-newbies

TABLE 9
Time-to-solution Comparison Against Other
Published Performance

Batch Size	epochs	Accuracy	hardware	time
256	100	58.7%	CPU + K20 GPU	144h
512	100	58.8%	DGX-1 station	6h 10m
4096	100	58.4%	DGX-1 station	2h 19m
32768	100	58.5%	512 KNLs	24m
32768	100	58.6%	1024 CPUs	11m

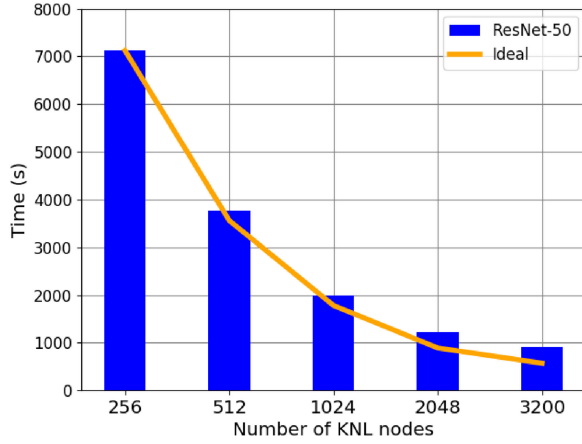


Fig. 17. Strong scaling performance of ResNet-50 with 32k batch size on KNL nodes.

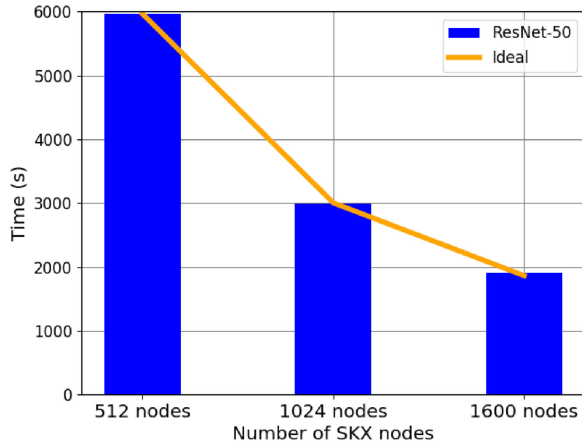


Fig. 18. Strong scaling performance of ResNet-50 with 32k batch size on SKX nodes.

The minute-level training performance is remarkable given the current practice, a comparison against known performance is presented in Table 9.

4.6 ImageNet training with ResNet-50

In this experiment, we use the ResNet-50 training case to evaluate the effectiveness of our approach in scalable DNN training. We use the ResNet-50 training on the ImageNet-1k dataset for 90 epochs with the batch size of 32k as the test case, run it at multiple scales on the KNL and SKX nodes on Stampede2, then compare the test accuracy and time-to-solution to the published results.

Figs. 17 and 18 show the strong scaling performance of the ResNet-50 case at various scales. Compared to AlexNet, ResNet-50 scales more efficiently to 1,024 KNL nodes and

TABLE 10
ResNet-50 Result. The DA means Data Augmentation

Batch	DA	epochs	accuracy	hardware	time
256	NO	90	73.0%	DGX-1 station	21h
256	YES	90	75.3%	16 KNLs	45h
8k	NO	90	72.7%	DGX-1 station	21h
8k	NO	90	72.7%	256 P100 GPUs	1h
8k	YES	90	75.3%	256 P100 GPUs	1h
16k	YES	90	75.3%	1024 SKX nodes	52m
16k	YES	90	75.3%	1600 SKX nodes	31m
32k	NO	90	72.6%	512 KNL nodes	1h
32k	YES	90	75.4%	512 KNL nodes	1h
32k	YES	90	75.4%	1024 SKX nodes	48m
32k	YES	90	74.2%	1600 SKX nodes	32m
32k	YES	90	75.4%	2048 KNL nodes	20m
32k	YES	90	76.4%	2048 KNL nodes	20m

The result of 76.4 percent accuracy is achieved by ResNet-50-v2.

TABLE 11
Comparison by 90-epoch ResNet50 Accuracy

Batch Size	256	8K	16K	32K	64K	DA
MSRA	75.3%	75.3%	—	—	—	weak
IBM	—	75.0%	—	—	—	—
SURFsara	—	75.3%	—	—	—	—
Facebook	76.3%	76.2%	75.2%	72.4%	66.0%	heavy
Our	73.0%	72.7%	72.7%	72.6%	70.0%	no
Our	75.3%	75.3%	75.3%	75.4%	73.2%	weak
Our	76.3%	76.6%	76.7%	76.4%	75.1%	heavy

DA means Data Augmentation.

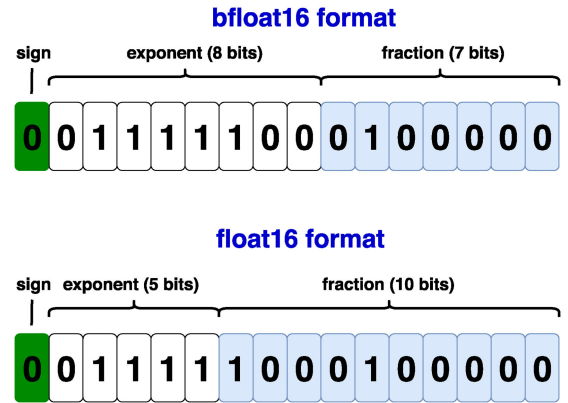


Fig. 19. The difference between bfloat16 and float16. Compared to float16 format, bfloat16 format has a wider range but a lower precision. bfloat16 format was supported in Google's TPU.

1,600 SKX nodes. This is because ResNet-50 has a relatively higher scaling ratio. In particular, the ResNet-50 case finishes in 32 minutes on 1,600 SKX nodes (3,200 Intel Xeon Platinum 8160 processors) and 20 minutes on 2,048 KNL nodes (2,048 Intel Xeon Phi 7250 processors).

A comprehensive result comparison against existing results is presented in Table 10. Codreanu et al. reported their experience on using Intel KNL clusters to speed up ImageNet-1k training by a blogpost.¹¹ They reported a 73.78 percent accuracy (with data augmentation) in less than 40 minutes on

11. <https://blog.surf.nl/en/imagenet-1k-training-on-intel-xeon-phi-in-less-than-40-minutes/>

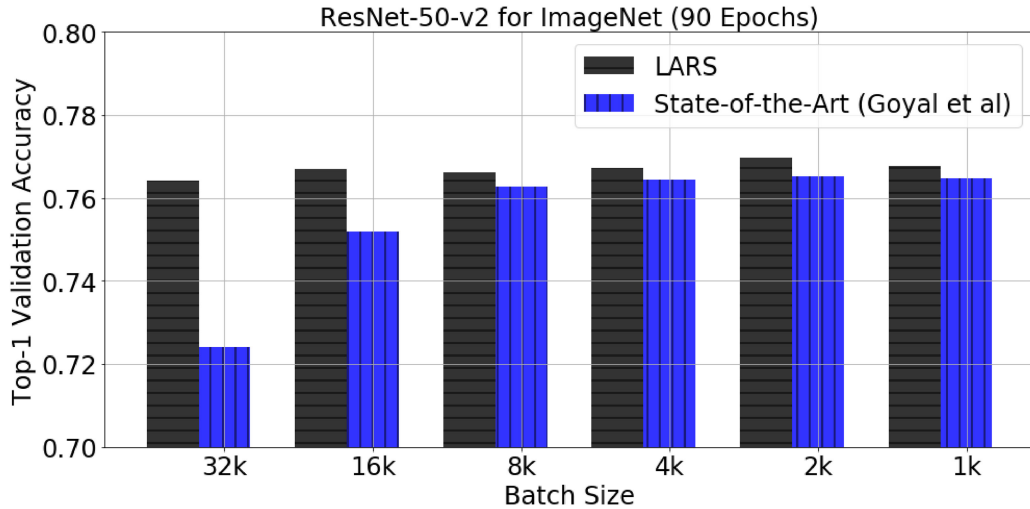


Fig. 20. Compared to Goyal et al. [1], our approach achieves a higher accuracy. Our approach is more constant and stable when the batch size is changing. Here, both Goyal et al. and us use the same ResNet-50 baseline (i.e., ResNet-50-v2).

512 KNL nodes with the batch size of 8k. However, this case only ran for 37 epochs. The complete 90-epoch training would take 80 minutes with a 75.25 percent accuracy.

Based on the original ResNet-50 model [26], we added data augmentation to our baseline. Our baseline is 75.3 percent top-1 test accuracy in 90 epochs. The model we used is available upon request. The test accuracy comparison is shown in Table 11. Although our baseline's accuracy is lower than Goyal et al., we achieve a correspondingly higher accuracy with batch sizes that are greater than 10k. When we use the same baseline, our results are better than Goyal et al. for all the batch sizes (Fig. 20).

4.7 Superlinear Speedup

For AlexNet training on eight NVIDIA P100 GPUs, we can achieve three times speedup. In this situation, the baseline and our approach used the same hardware. For ResNet-50 on distributed systems, the speedup is extremely large. Because the baseline's workload is limited to 256 samples at each iteration, it can only fully utilize 16 KNL nodes (Intel Xeon Phi 7250) or 1088 cores. Now our approach is able to process 32,768 samples each iteration, we can use 2048 KNL nodes or 139,264 cores. The perfect speedup is $139264/1088 = 128\times$. But we achieved a superlinear speedup of $45h/20m = 135\times$. The reason is that our communication overhead is much lower than the baseline (fewer messages and less volume of data moved over network), which is shown in Figs. 13 and 14. On the other hand, the single-node efficiency of our approach is higher (Fig. 6).

4.8 Implementation of LARS on Cloud TPUs¹²

Most of the emerging computer architectures support low-precision floating-point formats such as float16 format¹³ and bfloat16 format.¹⁴ The illustration of bfloat16 and float16 are

12. A paper titled *Image Classification at Supercomputer Scale* by Ying et al. was published after this paper was submitted. They use Google's internal TPU system, which is different from the cloud TPU system. Besides, they use a newer version of TPU systems. They acknowledged the first author of this paper has the contribution of implementing LARS solver on TPUs.

13. https://en.wikipedia.org/wiki/Half-precision_floating-point_format

14. https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

Authorized licensed use limited to: Peking University. Downloaded on July 13, 2026 at 19:07:08 UTC from IEEE Xplore. Restrictions apply.

shown in Fig. 19. As introduced in Section 2.5, we are interested in evaluating Google's TPU architecture. TPU's designers picked bfloat16 to support the low-precision format. In our experiments, we confirm that using bfloat16 rather than float16 is a good choice because most of the deep learning applications do not need a high precision to get the target application accuracy. To reduce the storage overhead and speed up the floating point operations, we implement our large-batch algorithm in bfloat16. All the weights, gradients, data, and activations in the deep neural networks training are represented in bfloat16 format. However, to avoid accuracy loss or even divergence, we store the hyper-parameters (e.g., learning rate, weight decay) into 32-bit single precision format. It is worth noting we only have fewer than ten hyper-parameters. Thus, the computation and storage costs of hyper-parameters are trivial. The computation and storage costs are dominated by the large matrices like weights, gradients, data, and activations. We tried a series of applications like AlexNet, GoogleNet, ResNet, and LSTM (using our large-batch algorithm), we confirm that we can achieve the results with the 32-bit single-precision CPU implementation. We write our code based on TensorFlow framework. We did some optimizations to reduce the synchronous communication overhead and improve the single-node performance.

The main deep neural networks are based on CNN structures and RNN structures. For CNN applications, we picked ImageNet training with ResNet-50. For RNN applications, we picked GNMT. We finished the Imagenet training with ResNet-50 on a cloud TPU (v2-8) in 7 hours and 50 minutes. Since one image requires 7.7 billion operations and the dataset has 1.28 million images, 90-epoch ImageNet training with ResNet-50 needs 8.87×10^{17} ($90 \times 1.28 \times 10^6 \times 7.7 \times 10^9$) operations. Thus, we achieved 31.5 TFlops on a cloud TPU for ResNet-50 training. LARS is an efficient algorithm that helps to improve the floating-point performance.

For GNMT, we are able to scale the batch size to 4K on a cloud TPU and get a BLEU score of 22.2. The application is out of memory on a cloud TPU when we increase the batch size more than 4K. So we stop the batch size scaling at 4K. The speed is 3637 samples per second. The sequence length

of the data is 150, which means the eight LSTM layers are unrolled to 1200 layers. Each GNMT layer is a 2048-by-4096 matrix and the input data is a 1-by-2048 vector. Thus, one input data sample requires 1200 matrix-vector multiplications. We achieved 33.3 TFlops on a cloud TPU for GNMT training. The baseline uses a batch size of 256, which only achieves the speed of 997.8 samples per second. Thus, we achieved around 4 times speedup on the same hardware.

4.9 Compared to State-of-the-art Implementation

To further improve our accuracy, we did some additional optimizations in our implementation. We implemented the strong data augmentation that was used in the system developed by Goyal et al. [1]. In this way, our baseline achieved the same accuracy with Goyal et al. [1]. By using the gradient clipping technique [36], we are able to achieve a higher accuracy for the baseline. Specifically, we use a mix of constant gradient clip and norm gradient clipping. We used label smoothing technique to improve the accuracy from 75.9 to 76.4 for batch size = 32K. Compared to the state-of-the-art implementation by Goyal et al. [1], our approach achieves a higher accuracy. Our approach is more constant and stable when the batch size is changing (Fig. 20).

5 CONCLUSION

In conclusion, we explore the large batch size approach to enable scalable DNN training on large scale computers. We examine the benefits and the challenges of this approach, and incorporate the LARS algorithm as the solution. We evaluate the implementation with the ImageNet-1k dataset and two neural network models of AlexNet and ResNet-50 at scale for the efficiency, test accuracy, training speed, and solution generality. Our solution is able to keep up with the baseline test accuracy for both test cases within the same number of epochs. We are able to reduce the ImageNet-1k training time from hours to minutes: With 1,024 SKX nodes, the AlexNet case finished in 11 minutes. While with 2,048 KNL nodes, the ResNet-50 case finished in 20 minutes. Our solution is general to be effective for both the AlexNet and ResNet-50 cases. We showcase large scale computers' capability in accelerating DNN training with massive computing resource with standard ImageNet-1k based benchmark. For GNMT, we achieved around 4 times speedup on the same hardware. We believe this is a pilot use case to motivate future DNN based research on large scale computers across domains in both industry and academia.

ACKNOWLEDGMENTS

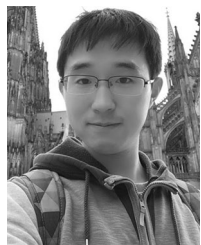
All the TPU results are based on the public cloud TPU system, we did not present any data related to Google's internal TPU system. The Layer-wise Adaptive Rate Scaling (LARS) algorithm (simulated on single node) was developed by Y. You, B. Ginsburg, and I. Gitman when Y. You was an intern at NVIDIA [6]. All the algorithms, implementations and experiments on distributed systems are done by the authors of this paper. We got the TPU access from Google's TFRC program (<https://www.tensorflow.org/tfrc>). We thank Alex Passos for providing a detailed introduction to AutoGraph in a talk. We also thank the discussions with Frank Chen, Sameer Kumar, David Patterson, Shawn Wang and Cliff Young. The

work presented in this paper was supported by the National Science Foundation, through the Stampede 2 (OAC-1540931) award. JD and YY are supported by the U.S. DOE Office of Science, Office of Advanced Scientific Computing Research, Applied Mathematics program under Award Number DE-SC0010200; by DARPA Award Number HR0011-12- 2-0016, ASPIRE Lab industrial sponsors and affiliates Intel, Google, HP, Huawei, LGE, Nokia, NVIDIA, Oracle and Samsung. Other industrial sponsors include Mathworks and Cray. In addition to ASPIRE sponsors, KK is supported by an auxiliary Deep Learning ISRA from Intel. CJH also thank XSEDE and Nvidia for independent support. We thank CSCS for granting us access to Piz Daint resources. The incomplete version of this paper was selected as the *best paper* (1 out of 313 submissions) at 47th International Conference on Parallel Processing (ICPP 2018), Eugene, Oregon, USA during August 13-16, 2018.

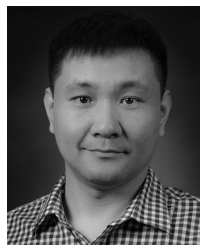
REFERENCES

- [1] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large mini-batch sgd: Training imagenet in 1 hour," arXiv:1706.02677, 2017.
- [2] Y. You, Z. Zhang, C.-J. Hsieh, J. Demmel, and K. Keutzer, "Imagenet training in minutes," in *Proc. 47th Int. Conf. Parallel Process.*, 2018, Art. no. 1.
- [3] D. Amodei, R. Anubhai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, J. Chen, M. Chrzanowski, A. Coates, G. Diamos, et al., "Deep speech 2: End-to-end speech recognition in english and mandarin," arXiv:1512.02595, 2015.
- [4] F. N. Iandola, K. Ashraf, M. W. Moskewicz, and K. Keutzer, "Firecaffe: Near-linear acceleration of deep neural network training on compute clusters," *CoRR*, vol. abs/1511.00175, 2015. [Online]. Available: <http://arxiv.org/abs/1511.00175>
- [5] F. N. Iandola, M. W. Moskewicz, K. Ashraf, and K. Keutzer, "Firecaffe: Near-linear acceleration of deep neural network training on compute clusters," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2592–2600.
- [6] Y. You, I. Gitman, and B. Ginsburg, "Scaling SGD batch size to 32K for imagenet training," 2017.
- [7] D. Das, S. Avancha, D. Mudigere, K. Vaidynathan, S. Sridharan, D. Kalamkar, B. Kaul, and P. Dubey, "Distributed deep learning using synchronous stochastic gradient descent," arXiv:1602.06709, 2016.
- [8] B. Recht, C. Re, S. Wright, and F. Niu, "Hogwild: A lock-free approach to parallelizing stochastic gradient descent," in *Proc. Advances Neural Inf. Process. Syst.*, 2011, pp. 693–701.
- [9] S. Zhang, A. E. Choromanska, and Y. LeCun, "Deep learning with elastic averaging SGD," in *Proc. Advances Neural Inf. Process. Syst.*, 2015, pp. 685–693.
- [10] P. H. Jin, Q. Yuan, F. Iandola, and K. Keutzer, "How to scale distributed deep learning?" arXiv:1611.04581, 2016.
- [11] I. Mitliagkas, C. Zhang, S. Hadjis, and C. Ré, "Asynchrony begets momentum, with an application to deep learning," in *Proc. 54th Annu. Allerton Conf. Commun. Control Comput.*, 2016, pp. 997–1004.
- [12] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, A. Senior, P. Tucker, K. Yang, Q. V. Le, et al., "Large scale distributed deep networks," in *Proc. Advances Neural Inf. Process. Syst.*, 2012, pp. 1223–1231.
- [13] F. Seide, H. Fu, J. Droppo, G. Li, and D. Yu, "On parallelizability of stochastic gradient descent for speech DNNs," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2014, pp. 235–239.
- [14] V. Codreanu, D. Podareanu, and V. Saeleto, "Scale out for large mini-batch SGD: Residual network training on imagenet-1k with improved accuracy and reduced time to train," arXiv:1711.04291, 2017.
- [15] T. Akiba, S. Suzuki, and K. Fukuda, "Extremely large minibatch SGD: Training resnet-50 on imagenet in 15 minutes," arXiv:1711.04325, 2017.
- [16] Y. You, Z. Zhang, J. Demmel, K. Keutzer, and C.-J. Hsieh, "Imagenet training in 24 minutes," arXiv:1709.05011, 2017.
- [17] J. L. Elman, "Finding structure in time," *Cognitive Sci.*, vol. 14, no. 2, pp. 179–211, 1990.
- [18] I. Goodfellow, Y. Bengio, A. Courville, and Y. Bengio, *Deep Learning*, vol. 1, Cambridge, MA, USA: MIT Press, 2016.
- [19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

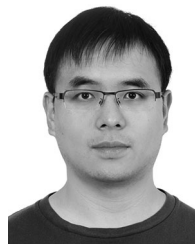
- [20] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, et al., "Google's neural machine translation system: Bridging the gap between human and machine translation," arXiv:1609.08144, 2016.
- [21] M.-T. Luong, E. Brevdo, and R. Zhao, "Neural machine translation (seq2seq) tutorial," 2017.
- [22] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, et al., "In-data-center performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Architecture*, 2017, pp. 1–12.
- [23] J. Chen, R. Monga, S. Bengio, and R. Jozefowicz, "Revisiting distributed synchronous SGD," arXiv:1604.00981, 2016.
- [24] S. Zhang, A. E. Choromanska, and Y. LeCun, "Deep learning with elastic averaging SGD," in *Proc. Advances Neural Inf. Process. Syst.*, 2015, pp. 685–693.
- [25] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," arXiv:1609.04836, 2016.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [27] A. Krizhevsky, "One weird trick for parallelizing convolutional neural networks," arXiv:1404.5997, 2014.
- [28] M. Li, "Scaling distributed machine learning with system and algorithm co-design," Ph.D. dissertation, The School of Computer Science (SCS) at Carnegie Mellon University, Santa Clara, CA, USA: Intel, 2017.
- [29] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," in *Proc. 22nd ACM Int. Conf. Multimedia*, 2014, pp. 675–678.
- [30] W. Gropp, E. Lusk, N. Doss, and A. Skjellum, "A high-performance, portable implementation of the MPI message passing interface standard," *Parallel Comput.*, vol. 22, no. 6, pp. 789–828, 1996.
- [31] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [32] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in mpich," *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, pp. 49–66, 2005.
- [33] R. Rabenseifner, "Optimization of collective reduction operations," in *Proc. Int. Conf. Comput. Sci.*, 2004, pp. 1–9.
- [34] R. Rabenseifner and J. L. Träff, "More efficient reduction algorithms for non-power-of-two number of processors in message-passing parallel systems," in *Proc. Eur. Parallel Virtual Mach. / Message Passing Interface Users Group Meeting*, 2004, pp. 36–46.
- [35] R. A. Vandegeijn, "On global combine operations," *J. Parallel Distrib. Comput.*, vol. 22, no. 2, pp. 324–328, 1994.
- [36] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1310–1318.



Yang You received the ME degree in computer science from Tsinghua University, in Jul. 2015. He is working toward the PhD degree at UC Berkeley Computer Science Division. His advisor is Prof. James Demmel. His research interests include Parallel Computing, Distributed Systems, and Machine Learning. He is a winner of IPDPS 2015 Best Paper award and ICPP 2018 Best Paper award. He is a Siebel Scholar and an ACM-IEEE CS George Michael Memorial HPC fellow. He is a member of the IEEE.



Zhao Zhang received the PhD degree in computer science from the University of Chicago. His advisor was Prof. Ian Foster. He is a research associate in Data Mining, Statistics, and HPC at Texas Advanced Computing Center, UT Austin. He is one of TACC's foremost experts in the application of advanced computing to AI. He did a post-doc at AMP Lab and the Berkeley Institute for Data Science (BIDS) at the University of California, Berkeley with Prof. Michael J. Franklin before joining TACC. He is a member of the IEEE.



ing new algorithms and optimization techniques for large-scale machine learning problems. Currently, he is working on developing new machine learning models as well as improving the model size, training speed, prediction speed, and robustness of popular (deep learning) models. He is a member of the IEEE.



of the ACM, IEEE and SIAM, and winner of the IEEE Computer Society Sidney Fernbach Award, the SIAM J. H. Wilkinson Prize in Numerical Analysis and Scientific Computing, IPDPS Charles Babbage Award, and numerous best paper prizes, including being the only 3-time winner of the SIAM Linear Algebra (SIAG/LA) Prize. He was an invited speaker at the 2002 International Congress of Mathematicians and the 2003 International Congress on Industrial and Applied Mathematics.



Kurt Keutzer received the PhD degree in computer science from Indiana University, in 1984, and then joined the research division of AT&T Bell Laboratories. He is a professor in the EECS Department of UC Berkeley. In 1991 he joined Synopsys, Inc. where he ultimately became chief Technical Officer and senior vice-president of Research. In 1998 he became professor of Electrical Engineering and Computer Science, University of California at Berkeley. His research group is currently focused on using parallelism to accelerate the training and Deployment of Deep Neural Networks for applications in computer vision, speech recognition, multi-media analysis, and computational finance. He has published six books, more than 250 refereed articles, and is among the most highly cited authors in Hardware and Design Automation. He was elected a fellow of the IEEE in 1996. His earlier research has had a lasting impact on Electronic Design Automation. For example, at the 50th Design Automation Conference he received a number of awards reflecting achievements more than the 50 year history of the conference. These awards included for Top Ten Cited Author, Top Ten Cited Paper and he was also recognized as among one of only three people to have received four Best Paper Awards in the history of the conference. While at Synopsys he worked with the General Counsel to co-develop the company's intellectual property strategy. Since joining Berkeley he has consulted directly with high-tech companies to do assessments of large (100+ patent) portfolios to assess their defensibility and monetary value. He has also worked, as time has allowed, as an IP consultant on individual patent infringement cases with many of Silicon Valley's top IP law firms. As an angel investor he has participated in the capital formation of twelve start-up companies including Coverity and Tensilica. Of these twelve, six have already seen profitable exits and two are still independent and growing. He has also served as an advisor to nine other start-ups including two that went on to become public companies. He is a fellow of the IEEE.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.